



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота №8
Технології розроблення програмного забезпечення
« Поведінкові патерни проектування: Memento, State, Strategy.»
«*System activity monitor*»

Виконав
студент групи ІА–32:
Феклістов Д.С.

Київ 2025

1. Мета роботи

Вивчити призначення та структуру поведінкових патернів проектування (Memento, State, Strategy).

Практично реалізувати патерн Memento (Знімок) у системі моніторингу для створення механізму збереження поточних показників системи (CPU, RAM) та їх подальшого відновлення ("Точки відновлення").

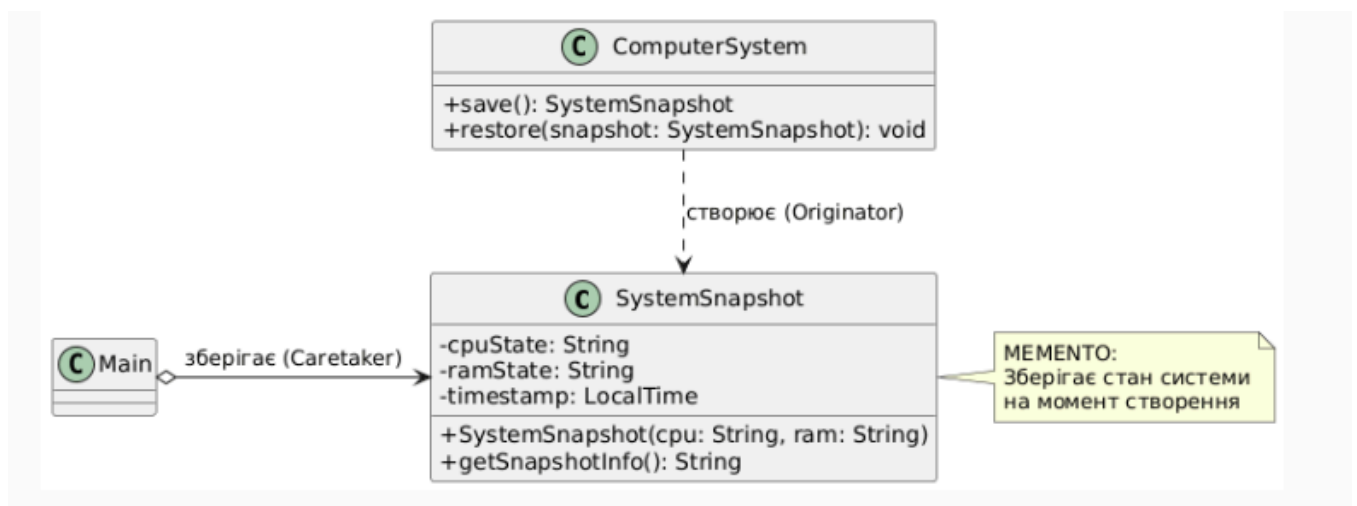
2. Теоретичні відомості

Memento (Знімок) — це поведінковий патерн проектування, який дозволяє зберігати та відновлювати попередній стан об'єкта, не розкриваючи подробиць його внутрішньої реалізації (не порушуючи інкапсуляцію).

Основні учасники патерна:

1. **Originator (Творець):** Об'єкт, стан якого потрібно зберігати. Створює знімок свого стану та може відновитися з нього. У нашому випадку — це фасад `ComputerSystem`.
2. **Memento (Знімок):** Об'єкт, що зберігає стан Творця. Він є незмінним (immutable) і не дозволяє іншим об'єктам змінювати збережені дані. У нашому випадку — `SystemSnapshot`.
3. **Caretaker (Опікун):** Відповідає за зберігання знімків, але не має права втручатися в їхній вміст. У нашому випадку — `Main`.

3. Діаграма класів (UML)



4. Хід роботи

1. **Створено пакет `monitor.memento`** та клас `SystemSnapshot`. Цей клас зберігає рядкові дані про стан процесора та пам'яті, а також час створення знімка. Поля класу оголошено як `final`.
2. **Модифіковано клас `ComputerSystem (Originator)`.**
 - Реалізовано метод `save()`, який зчитує поточні дані з сенсорів і повертає об'єкт `SystemSnapshot`.
 - Реалізовано метод `restore(SystemSnapshot snapshot)`, який приймає знімок і виводить збережену інформацію, імітуючи відновлення стану.
3. **Оновлено клас `Main (Caretaker)`.** Реалізовано сценарій демонстрації:
 - Перевірка поточного стану.
 - Створення точки відновлення (Save).
 - Зміна стану системи (імітація роботи/навантаження через паузу).
 - Відновлення стану з точки відновлення (Restore) і порівняння даних.

5. Код програми

Файл: `src/monitor/memento/SystemSnapshot.java`

```
package monitor.memento;

import java.time.LocalDateTime;

// MEMENTO: Зберігає стан системи в певний момент часу
public class SystemSnapshot {
    private final String cpuState;
    private final String ramState;
    private final LocalDateTime timestamp;

    public SystemSnapshot(String cpuState, String ramState)
    {
        this.cpuState = cpuState;
        this.ramState = ramState;
        this.timestamp = LocalDateTime.now();
    }

    public String getCpuState() {
        return cpuState;
    }
}
```

```

    public String getRamState() {
        return ramState;
    }

    public String getSnapshotInfo() {
        return String.format("[%s] CPU: %s | RAM: %s",
timestamp, cpuState, ramState);
    }
}

```

Фрагмент src/monitor/facade/ComputerSystem.java

```

// 1. Створити точку відновлення (Save)
public SystemSnapshot save() {
    logger.log("[MEMENTO] Збереження точки
відновлення...");
    // Ми беремо поточні дані з сенсорів і пакуємо їх у
знімок
    return new SystemSnapshot(
        cpu.getCpuInfo(),
        memory.getMemoryInfo()
    );
}

// 2. Відновити стан (Restore)
public void restore(SystemSnapshot snapshot) {
    logger.log("[MEMENTO] Завантаження точки
відновлення...");
    System.out.println("    >>> ВІДНОВЛЕНО ДАНІ: " +
snapshot.getSnapshotInfo());

    // Тут ми могли б "відкотити" налаштування, якщо
вони у нас були.
    // Але оскільки це моніторинг, ми просто показуємо
збережені дані.
}
}

```

Фрагмент src/monitor/Main.java

```
// 1. Початковий стан
System.out.println("--- КРОК 1: Поточний стан ---");
computer.checkSystem();
Thread.sleep(1500); // Час на зміну параметрів

// 2. ЗБЕРЕЖЕННЯ (Створюємо точку відновлення)
System.out.println("\n--- КРОК 2: Створення бекапу
(Save) ---");
SystemSnapshot backup = computer.save();

// 3. Зміна стану (імітуємо навантаження)
System.out.println("\n--- КРОК 3: Робота системи (зміна
даних) ---");
Thread.sleep(2000); // Дані CPU зміняться
computer.checkSystem();

// 4. ВІДНОВЛЕННЯ (Повертаємося до старого запису)
System.out.println("\n--- КРОК 4: Відкат системи
(Restore) ---");
computer.restore(backup);
}
```

6. Результати виконання

```
=== SYSTEM MONITOR (Lab 9: Memento) ===

-> [LOG]: Ініціалізація файлової системи логів...
--- КРОК 1: Поточний стан ---
[03:24:51] [ФАСАД] Поточний стан:
[03:24:51] CPU: Ядер - 12, Загрузка - 0,0%, Температура - 51°C
[03:24:51] RAM: 11,7 GB / 13,9 GB | HDD: 56,0 GB / 475,7 GB

--- КРОК 2: Створення бекапу (Save) ---
[03:24:53] [MEMENTO] Збереження точки відновлення...

--- КРОК 3: Робота системи (зміна даних) ---
[03:24:55] [ФАСАД] Поточний стан:
[03:24:55] CPU: Ядер - 12, Загрузка - 2,6%, Температура - 50°C
[03:24:55] RAM: 11,7 GB / 13,9 GB | HDD: 56,0 GB / 475,7 GB

--- КРОК 4: Відкат системи (Restore) ---
[03:24:55] [MEMENTO] Завантаження точки відновлення...
>>> ВІДНОВЛЕНО ДАНІ: [03:24:53.148238] CPU: CPU: Ядер - 12, Загрузка - 18,5%, Температура - 68°C | RAM: RAM: 11,7 GB / 13,9 GB | HDD: 56,0 GB / 475,7 GB
```

7. Висновок

В ході лабораторної роботи було успішно реалізовано поведінковий патерн **Memento (Знімок)**. Це дозволило додати до системи моніторингу функціонал створення контрольних точок (snapshots). Реалізація забезпечує збереження стану системи в окремий об'єкт без розкриття внутрішньої структури цього стану, що відповідає принципам інкапсуляції об'єктно-орієнтованого програмування.

8. Відповіді на контрольні питання

1. Яке призначення патерну Memento?
Збереження та винесення внутрішнього стану об'єкта за його межі для подальшого відновлення, не порушуючи інкапсуляцію.
2. Яке призначення патерну State?
Дозволяє об'єкту змінювати свою поведінку залежно від свого внутрішнього стану. Ззовні здається, що змінився клас об'єкта.
3. Яке призначення патерну Strategy?
Визначає сімейство алгоритмів, інкапсулює кожен з них в окремий клас і робить їх взаємозамінними. Дозволяє змінювати алгоритм незалежно від клієнта.
4. Хто такий Originator у патерні Memento?
Це об'єкт, стан якого ми хочемо зберегти (у нашій роботі — ComputerSystem).